ENGLISH PANAMÁ · EQUIPO DE DESARROLLO

Pruebas del servidor con tu propia base de datos en Docker

Guía para todo el equipo · Noviembre de 2026 · Tiempo de montaje: 10 a 15 minutos la primera vez

1. Para qué sirve y por qué es obligatorio

Parte de las pruebas del servidor (`server/tests`) necesita una base de datos MySQL real: crean y borran usuarios, docentes, estudiantes y cuentas de gestión, y escriben filas en la auditoría. **Nunca se corren contra producción**: ensuciarían los datos reales, y la auditoría es de solo-inserción, por lo que esas filas no se pueden borrar después.

La solución es que cada persona tenga **su propia base desechable en su computador**, dentro de un contenedor Docker: la misma versión de MySQL que producción (8.0), aislada, que se crea en un minuto y se puede destruir y rehacer cuando quieras. Así puedes probar tu código contra una base de datos **antes de abrir un pull request**, sin tener ningún acceso a la base final.

> **Regla de oro**
> Las pruebas escriben en la base a la que apunten las variables `DB_*` y `AUDIT_DB_*`. Antes de correrlas, comprueba siempre que `DB_HOST` sea `127.0.0.1`. Si alguna vez apunta a otra dirección, **no corras nada**.

2. Qué necesitas

Herramienta	Detalle
Docker Desktop	Windows, macOS o Linux (en Linux basta Docker Engine). Debe estar abierto y con el motor en marcha (el ícono de la ballena sin avisos).
Node.js 20 o superior	El CI usa Node 20. Comprueba con `node -v`.
Git y el repositorio	Clonado, con tu rama de trabajo actualizada desde `develop`.
Puerto 3306 libre	El servidor **solo** habla con la base de auditoría por el puerto estándar. Si tienes MySQL instalado en tu equipo y ocupa el 3306, detenlo mientras pruebas.
Espacio	Unos 600 MB para la imagen de MySQL.

Todos los comandos de esta guía se escriben **desde la raíz del repositorio**, salvo los de Node, que se ejecutan dentro de `server/`. Los comandos de Docker son idénticos en PowerShell, Git Bash, macOS y Linux; solo cambian las variables de entorno (paso 5), y ahí se dan las dos versiones.

3. Montaje paso a paso (una sola vez)

1 Crear el contenedor de MySQL

```
docker run -d --name ep-test-mysql -p 127.0.0.1:3306:3306 -e MYSQL_ROOT_PASSWORD=ep-local-root -e MYSQL_DATABASE=englishpanama mysql:8.0 --default-time-zone=+00:00 --character-set-server=utf8mb4 --collation-server=utf8mb4_0900_ai_ci
```

`127.0.0.1:` delante del puerto hace que la base solo sea visible desde tu propio equipo. La zona `+00:00` (UTC) es la misma que usa el código. La clave `ep-local-root` es solo para esta base desechable: **no reutilices contraseñas reales**.

2 Esperar a que MySQL esté listo

La primera vez tarda entre 30 y 60 segundos: MySQL se inicializa, se reinicia solo y recién entonces acepta conexiones. Repite este comando hasta que muestre `1` (si dice *Can't connect*, espera unos segundos y vuelve a intentarlo):

```
docker exec ep-test-mysql mysql -uroot -pep-local-root -h127.0.0.1 -e "SELECT 1"
```

> **Por qué `-h127.0.0.1`**
> Durante el arranque inicial existe un servidor temporal que solo acepta conexiones locales por socket. Probar por TCP (`-h127.0.0.1`) confirma que ya está el servidor definitivo. Un `mysqladmin ping` sin ese parámetro puede dar un falso "listo".

3 Crear la base de auditoría y los usuarios

Esto reproduce los permisos de producción: la aplicación tiene control total sobre su base, pero el usuario de auditoría solo puede **insertar y leer**.

```
docker exec ep-test-mysql mysql -uroot -pep-local-root -e "CREATE DATABASE IF NOT EXISTS englishpanama_audit CHARACTER SET utf8mb4; CREATE USER IF NOT EXISTS 'app_user'@'%' IDENTIFIED BY 'ep-local-app'; GRANT ALL ON englishpanama.* TO 'app_user'@'%'; CREATE USER IF NOT EXISTS 'audit_writer'@'%' IDENTIFIED BY 'ep-local-audit'; FLUSH PRIVILEGES;"
```

4 Cargar el esquema y las migraciones

Copia los archivos SQL dentro del contenedor y cárgalos en este orden:

```
docker cp server/database ep-test-mysql:/tmp/db

# Base principal: esquema, cuentas de gestión y migraciones
docker exec ep-test-mysql sh -c "mysql -uroot -pep-local-root englishpanama < /tmp/db/schema.sql"
docker exec ep-test-mysql sh -c "mysql -uroot -pep-local-root englishpanama < /tmp/db/staff.sql"
docker exec ep-test-mysql sh -c "mysql -uroot -pep-local-root englishpanama <
/tmp/db/migrations/phase2-lockout.sql"
docker exec ep-test-mysql sh -c "mysql -uroot -pep-local-root englishpanama <
/tmp/db/migrations/phase2-platform-mfa.sql"
docker exec ep-test-mysql sh -c "mysql -uroot -pep-local-root englishpanama <
/tmp/db/migrations/phase3-must-change-password.sql"
docker exec ep-test-mysql sh -c "mysql -uroot -pep-local-root englishpanama <
/tmp/db/migrations/phase4-class-grade.sql"
docker exec ep-test-mysql sh -c "mysql -uroot -pep-local-root englishpanama <
/tmp/db/migrations/phase5-content-review.sql"
docker exec ep-test-mysql sh -c "mysql -uroot -pep-local-root englishpanama <
/tmp/db/migrations/phase6-totp-replay.sql"

# Base de auditoría
docker exec ep-test-mysql sh -c "mysql -uroot -pep-local-root englishpanama_audit <
/tmp/db/audit.sql"
docker exec ep-test-mysql mysql -uroot -pep-local-root -e "GRANT INSERT, SELECT ON
englishpanama_audit.audit_log TO 'audit_writer'@'%'; FLUSH PRIVILEGES;"
```

Se ve un aviso "*Using a password on the command line interface can be insecure*": es normal y aquí no importa (es una base local desechable). Si algún comando muestra un `ERROR` de verdad, no sigas: revisa la sección 7.

> **Qué NO cargar**
> No ejecutes `migrations/phase6-hardening.sql` tal cual (tiene una restricción que choca con el flujo de revisión de contenido) ni los `prod-upgrade-*.sql` (son para actualizar producción, no para una base nueva).

> **Cuando aparezca una migración nueva**
> Si un compañero agrega un archivo en `server/database/migrations/`, vuelve a ejecutar `docker cp` y cárgalo con el mismo comando. Las migraciones son idempotentes: repetir una no rompe nada.

5 Variables de entorno (en cada terminal donde vayas a probar)

Las pruebas leen la configuración del entorno, no de un archivo. **No hace falta (ni conviene) crear un `.env` para esto.**

POWERSHELL (WINDOWS)

GIT BASH, MACOS, LINUX

```
$env:DB_HOST="127.0.0.1"
$env:DB_PORT="3306"
$env:DB_USER="app_user"
$env:DB_PASSWORD="ep-local-app"
$env:DB_NAME="englishpanama"
$env:AUDIT_DB_HOST="127.0.0.1"
$env:AUDIT_DB_USER="audit_writer"
$env:AUDIT_DB_PASSWORD="ep-local-audit"
$env:AUDIT_DB_NAME="englishpanama_audit"
$env:NODE_ENV="test"
$env:JWT_SECRET="local-jwt-secret-local-jwt-
secret"
$env:STAFF_JWT_SECRET="local-staff-secret-
local-staff-secret"
$env:STAFF_MFA_KEY = node -e
"process.stdout.write(require('crypto').randomB
ytes(32).toString('base64'))"
```

```
export DB_HOST=127.0.0.1
export DB_PORT=3306
export DB_USER=app_user
export DB_PASSWORD=ep-local-app
export DB_NAME=englishpanama
export AUDIT_DB_HOST=127.0.0.1
export AUDIT_DB_USER=audit_writer
export AUDIT_DB_PASSWORD=ep-local-audit
export AUDIT_DB_NAME=englishpanama_audit
export NODE_ENV=test
export JWT_SECRET=local-jwt-secret-local-jwt-
secret
export STAFF_JWT_SECRET=local-staff-secret-
local-staff-secret
export STAFF_MFA_KEY="$(node -e
"process.stdout.write(require('crypto').randomB
ytes(32).toString('base64'))")"
```

`STAFF_MFA_KEY` debe ser una clave de **32 bytes en base64** (44 caracteres). Se genera al vuelo y sirve para toda la sesión de esa terminal. Cada terminal nueva necesita sus variables otra vez.

> **Comprobación rápida**
> En PowerShell: `$env:DB_HOST` · en Bash: `echo $DB_HOST`. Debe mostrar `127.0.0.1`.

6 Instalar dependencias y correr las pruebas

```
cd server
npm ci
npm test
```

La suite completa tarda unos **2 minutos**: `npm test` la ejecuta en serie (`--test-concurrency=1`), porque varios archivos comparten la misma base y algunas pruebas se interfieren si corren en paralelo. Para correr un solo archivo (más rápido mientras desarrollas):

```
node --test tests/class-grade.test.js
```

Cómo se ve el éxito

```
ℹ tests 323
ℹ pass 321
ℹ fail 0
ℹ skipped 2
```

Los números exactos cambian cuando se agregan pruebas. Lo que importa: `fail 0` y **muy pocos** `skipped` (hoy 2, ambas en `api.test.js`, ya conocidas). Si ves decenas de pruebas omitidas, las pruebas no encontraron la base: revisa variables y contenedor.

Atajo: los pasos 1 a 4 con un solo comando

El repositorio incluye un script que hace los pasos 1 a 4 por ti y deja la base lista (tarda 1 a 2 minutos). Si no quieres teclear todo lo anterior, usa este atajo; los pasos manuales siguen siendo la referencia de lo que el script hace.

GIT BASH, MACOS, LINUX

```
bash server/scripts/setup-test-db.sh
eval "$(bash server/scripts/setup-test-db.sh --env)"
```

POWERSHELL (WINDOWS)

```
.\server\scripts\setup-test-db.ps1
.\server\scripts\setup-test-db.ps1 -Env | Invoke-Expression
```

La primera línea crea la base; la segunda define las variables de entorno en la terminal actual (el paso 5). Si el contenedor ya existe, el script se detiene y avisa; añade `--reset` (Bash) o `-Reset` (PowerShell) para borrarlo y rehacerlo limpio. Si PowerShell dice que la ejecución de scripts está deshabilitada, ejecuta `powershell -ExecutionPolicy Bypass -File .\server\scripts\setup-test-db.ps1`. Después sigue en el paso 6 (`npm ci` y `node --test`).

4. Uso diario

Quiero...	Comando
Apagar la base al terminar el día	`docker stop ep-test-mysql`
Volver a encenderla (los datos se conservan)	`docker start ep-test-mysql` (esperar unos segundos)
Ver si está corriendo	`docker ps`
Empezar de cero, con la base limpia	`docker rm -f ep-test-mysql` y repetir los pasos 1 a 4
Mirar tablas o datos a mano	`docker exec -it ep-test-mysql mysql -uroot -pep-local-root englishpanama`
Ver los registros de MySQL si algo falla	`docker logs ep-test-mysql`

Las pruebas crean filas de ejemplo y las limpian, pero una prueba interrumpida puede dejar datos sueltos. Si ves fallos raros que no tienen que ver con tu cambio, **rehacer la base desde cero** (cuarta fila) es la primera solución y toma pocos minutos.

5. Antes de abrir un pull request

1. Tu rama está al día con `develop` y aplicaste en tu base las migraciones nuevas que hayan llegado.
2. `npm test` termina con **0 fallos**. Anota el resultado en la descripción del PR.
3. Si tu cambio toca autenticación, permisos, límites de intentos, el reto diario o la base de datos, **agrega una prueba que hable con la base** y que falle sin tu corrección. Las pruebas de funciones puras no detectan estos problemas.
4. Si cambias la estructura de la base: crea una migración idempotente en `server/database/migrations/` **y** actualiza `schema.sql` (para que una base nueva quede igual que producción) y la guía de producción.
5. No subas `.env`, claves ni contraseñas, ni siquiera las "de prueba" de esta guía en otros archivos.

6. Lo que esto no cubre

- **El cliente (React):** estas pruebas son del servidor. La interfaz se prueba aparte, en el navegador.

- **Diferencias con producción:** la base local es nueva y vacía; no tiene el volumen de datos ni la zona horaria de las máquinas virtuales. Un cambio de migraciones que toque datos existentes debe probarse además sobre una base con datos de ejemplo.
- **Varios procesos o carga real:** la suite no mide rendimiento ni concurrencia entre varios servidores.

Pasar la suite es condición necesaria para abrir un PR, **no es suficiente**: el PR sigue pasando por revisión.

7. Problemas frecuentes

Síntoma	Causa y solución
`port is already allocated` al crear el contenedor	Otro programa usa el 3306 (un MySQL local u otro contenedor). Deténlo o, si es un contenedor viejo de esta guía, bórralo con `docker rm -f ep-test-mysql`.
`Can't connect to local MySQL server`	MySQL todavía está arrancando. Espera y repite el paso 2.
`Access denied for user 'app_user'`	Falta el `GRANT` del paso 3, o `DB_PASSWORD` no coincide con `ep-local-app`. Repite el paso 3 y revisa las variables.
`Unknown database 'englishpanama_audit'`	Se ejecutó `audit.sql` antes de crear la base. Repite el paso 3 y luego el bloque de auditoría del paso 4.
`Unknown column …` en las pruebas	Falta una migración. Repite el paso 4 con los archivos actuales de `server/database/migrations/`.
Decenas de pruebas "omitidas" (`skipped`)	Las pruebas no encontraron la base o faltan variables. Revisa que el contenedor esté en marcha y que la terminal tenga las variables del paso 5.
`STAFF_MFA_KEY debe ser de 32 bytes…`	La variable está vacía o mal generada. Vuelve a ejecutar el último comando del paso 5 en esa terminal (debe quedar de 44 caracteres).
La suite se queda colgada	Interrúmpela con Ctrl + C y córrela con límite de tiempo: `timeout 600 npm test` (Git Bash, macOS, Linux). Si persiste, corre archivo por archivo para encontrar cuál.
Docker Desktop no arranca	Ábrelo manualmente y revisa si pide aceptar una actualización o reiniciar. En Windows puede ayudar `wsl --shutdown` y abrirlo de nuevo.

8. Resumen en cuatro líneas

1. Crea el contenedor `ep-test-mysql` y espera a que acepte conexiones.
2. Crea los usuarios y carga `schema.sql`, `staff.sql`, las migraciones y `audit.sql`.
3. Define las variables de entorno apuntando a `127.0.0.1` y corre `npm test` dentro de `server/`.
4. Abre el PR solo con **0 fallos**, y cuenta el resultado en la descripción.
